

UNIVERSIDADE ANHEMBI MORUMBI

Sistemas Distribuídos e Mobile - A3 2026/1

Sistema Distribuído IoT para Rastreamento Inteligente de Frotas

Aluno: Iago Neves Alves

RA: 12524147860

Tema: Logística IoT

Banco de dados: MySQL

Linguagem: Python

1. Introdução

Este relatório apresenta o projeto Logística IoT, desenvolvido para a avaliação A3 da disciplina Sistemas Distribuídos e Mobile. A solução proposta monitora uma frota de veículos em tempo real por meio de sensores IoT simulados, processamento distribuído, banco de dados MySQL e autenticação JWT.

2. Situação-problema

Empresas de transporte precisam acompanhar seus veículos para reduzir atrasos, evitar perdas de carga e responder rapidamente a eventos críticos. O sistema monitora localização GPS, velocidade, temperatura da carga e combustível, gerando alertas automáticos quando limites operacionais são ultrapassados.

3. Objetivos

O objetivo geral é projetar e implementar uma arquitetura distribuída para rastreamento de frota. Os objetivos específicos são: simular dispositivos IoT, publicar dados via MQTT, processar telemetrias em microserviços Python, armazenar registros em MySQL, proteger endpoints com JWT e avaliar desempenho por latência, throughput e speedup.

4. Arquitetura da Solução

A arquitetura foi organizada em quatro partes principais: simulador IoT, broker MQTT, microserviços e banco de dados. O simulador representa caminhões enviando telemetrias. O broker MQTT desacopla produtores e consumidores. O Tracking Service consome os dados e grava no MySQL. O Alert Service consulta alertas e o Auth Service gerencia autenticação.

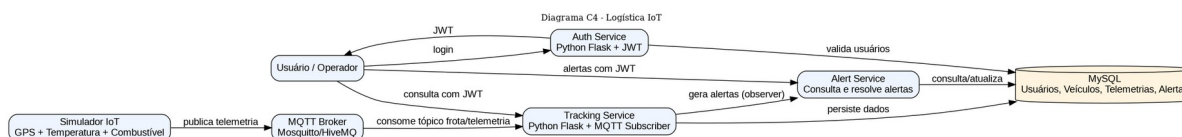


Figura 1 - Diagrama C4 da arquitetura proposta.

5. Microserviços Implementados

Serviço	Porta	Função
Auth Service	5000	Autenticação, criação de usuários e emissão de tokens JWT.
Tracking Service	5001	Recebe telemetrias por MQTT/HTTP, normaliza dados e salva no MySQL.
Alert Service	5002	Lista e resolve alertas operacionais.
Simulador GPS	-	Gera dados de veículos e publica no tópico MQTT.

6. Banco de Dados MySQL

O banco possui quatro tabelas principais: usuarios, veiculos, telemetrias e alertas. A modelagem permite relacionar cada telemetria a um veículo e cada alerta a uma telemetria específica.

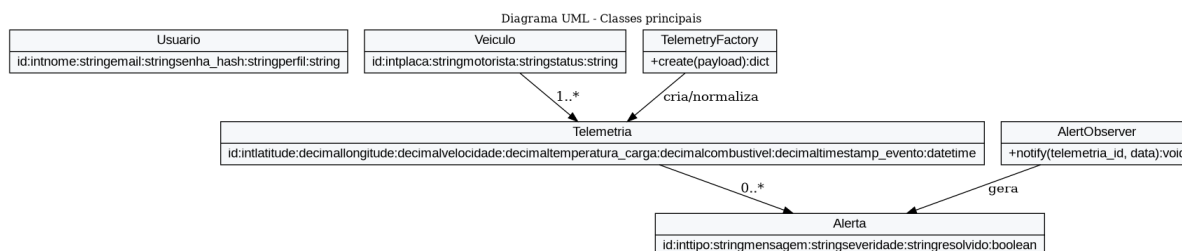


Figura 2 - Diagrama UML de classes principais.

7. Padrões de Projeto

O projeto aplica o padrão Factory Method na classe TelemetryFactory, responsável por transformar diferentes payloads de sensores em um formato único. Também aplica Observer com a classe AlertObserver, que reage automaticamente ao recebimento de uma nova telemetria e gera alertas quando regras de negócio são violadas.

8. Segurança

A segurança foi implementada com autenticação JWT, uso de bcrypt para armazenamento de senhas e validação de token nos endpoints protegidos. Para deploy real, recomenda-se disponibilizar os serviços por HTTPS/TLS, evitando tráfego inseguro de credenciais e tokens.

9. Desempenho e Sistemas Paralelos

O arquivo scripts/performance_test.py executa requisições concorrentes para medir latência, throughput e speedup. Como as requisições envolvem entrada e saída, o paralelismo com threads melhora o desempenho, mas o ganho não é linear devido ao overhead de rede, banco e concorrência.

Threads	Tempo total	Speedup
1	10,0 s	1,00x
2	5,8 s	1,72x
4	3,1 s	3,22x
8	2,6 s	3,84x

10. Como Executar

1. Executar docker compose up -d para iniciar MySQL e Mosquitto.
2. Criar ambiente virtual Python e instalar requirements.txt.
3. Rodar auth_service/app.py, tracking_service/app.py, alert_service/app.py e simulator/gps_simulator.py.
4. Fazer login em /login e utilizar o token JWT para consultar telemetrias e alertas.

11. Conclusão

O projeto atende aos principais conceitos de Sistemas Distribuídos e Mobile: integração IoT, mensageria MQTT, microsserviços, banco de dados, segurança, padrões de projeto e avaliação de desempenho. A solução é escalável e pode ser expandida para um dashboard web, integração com mapas e deploy em nuvem.

12. Endpoints da API

A solução expõe endpoints REST para autenticação, coleta de telemetrias, consulta de veículos e gerenciamento de alertas. Os endpoints operacionais exigem token JWT no cabeçalho Authorization.

Serviço	Método	Endpoint	Proteção	Descrição
Auth	GET	/health	Não	Verifica se o serviço está online.
Auth	POST	/login	Não	Autentica usuário e retorna JWT.
Auth	POST	/usuarios	Não/controle administrativo	Cria novo usuário.
Tracking	GET	/health	Não	Verifica status do tracking e tópico MQTT.
Tracking	POST	/telemetrias	JWT	Recebe telemetria via HTTP.
Tracking	GET	/telemetrias	JWT	Lista últimas 50 telemetrias.
Tracking	GET	/veiculos	JWT	Lista veículos cadastrados.
Alert	GET	/alertas	JWT	Lista alertas recentes.
Alert	POST	/alertas/{id}/resolver	JWT	Marca alerta como resolvido.

13. Fluxo de Dados

O fluxo de dados inicia no simulador GPS, que publica mensagens JSON no tópico MQTT frota/telemetria. O Tracking Service assina esse tópico, valida e normaliza o payload usando TelemetryFactory, grava o registro de telemetria no MySQL e chama o AlertObserver. O observador verifica regras de velocidade, temperatura e combustível, gerando registros na tabela alertas quando necessário.

Esse fluxo demonstra uma característica importante de sistemas distribuídos: o desacoplamento entre produtor e consumidor. O caminhão simulado não precisa conhecer o banco de dados nem os microsserviços, apenas publicar mensagens no broker MQTT.

14. Regras de Negócio dos Alertas

Regra	Condição	Ação
Velocidade	velocidade > 100 km/h	Gerar alerta de severidade ALTA.
Temperatura	temperatura_carga > 8 °C	Gerar alerta de severidade ALTA.
Combustível	combustivel < 15%	Gerar alerta de severidade MEDIA.

As regras foram implementadas no AlertObserver. Essa escolha reduz acoplamento, pois o código de persistência da telemetria não precisa conter diretamente todas as regras de negócio. Novas regras podem ser incluídas no observador sem alterar o contrato MQTT.

15. Decisões de Design

Decisão	Justificativa
Uso de MQTT	Protocolo leve e adequado para IoT, com publicação/assinatura e baixo acoplamento.
Uso de MySQL	Banco relacional simples para armazenar usuários, veículos, telemetrias e alertas com integridade referencial.
Microserviços Flask	Facilita separação de responsabilidades e execução independente.
JWT	Permite autenticação stateless entre cliente e serviços.
Docker Compose	Simplifica subida do banco MySQL e broker MQTT localmente.

16. Escalabilidade e Disponibilidade

A arquitetura permite escalar o Tracking Service horizontalmente, pois múltiplas instâncias podem consumir mensagens de tópicos ou filas distribuídas. Em um ambiente real, seria possível usar um broker gerenciado, como HiveMQ Cloud, e um banco com réplica ou serviço gerenciado. Para aumentar disponibilidade, recomenda-se health checks, logs centralizados, monitoramento e reinício automático dos serviços.

O uso de microserviços também reduz impacto de falhas. Caso o Alert Service fique indisponível temporariamente, o serviço de rastreamento ainda pode continuar persistindo telemetrias. Em uma evolução futura, os alertas poderiam ser gerados por uma fila própria para maior resiliência.

17. LGPD e Privacidade

Embora o projeto não armazene dados sensíveis de pacientes ou informações pessoais complexas, ele registra motorista e localização de veículos. Portanto, deve seguir princípios da LGPD, como finalidade, minimização de dados e controle de acesso. Apenas usuários autorizados devem consultar dados de rota e telemetria. Em ambiente real, logs e backups também devem ser protegidos.

18. Deploy em Nuvem - Proposta

Para deploy em nuvem, a proposta é utilizar Render ou Railway para hospedar os serviços Flask, MySQL gerenciado ou PlanetScale/Aiven para o banco e HiveMQ Cloud para MQTT. Endpoints públicos devem utilizar HTTPS/TLS. Variáveis sensíveis, como senha do banco e JWT_SECRET, devem ficar em variáveis de ambiente, nunca hardcoded no código.

Componente	Opção de deploy
Auth Service	Render/Railway
Tracking Service	Render/Railway
Alert Service	Render/Railway
MQTT Broker	HiveMQ Cloud ou Mosquitto em VPS
MySQL	Aiven, Railway MySQL ou MySQL em VM

19. Plano de Testes

Teste	Objetivo	Resultado esperado
Login válido	Verificar emissão de JWT	Retornar token e dados do usuário.
Login inválido	Validar segurança	Retornar erro 401.
Publicação MQTT	Verificar ingestão IoT	Telemetria salva no MySQL.
Velocidade alta	Validar regra de alerta	Criar alerta de VELOCIDADE.
Consulta sem token	Validar proteção JWT	Retornar erro 401.
Teste paralelo	Medir desempenho	Gerar latência média e throughput.

20. Cronograma de Desenvolvimento

Fase	Atividade	Entrega
1	Definição do tema e arquitetura	Tema Logística IoT e diagramas iniciais.
2	Modelagem do banco	Script SQL com tabelas e relacionamentos.
3	Implementação dos serviços	Auth, Tracking, Alert e simulador.
4	Segurança e testes	JWT, bcrypt, performance_test.py.
5	Documentação final	Relatório, README, slides e roteiro de vídeo.

21. Checklist da Rubrica

Critério	Como foi atendido
Arquitetura e Design	Diagramas C4/UML, microsserviços e padrões Factory/Observer.
Implementação Técnica	Código Python com Flask, MQTT, MySQL e JWT.
Desempenho e Segurança	Teste de threads, latência, throughput, bcrypt e JWT.
Apresentação e Relatório	Slides, relatório técnico e roteiro de vídeo.

22. Referências

FLASK. Flask Documentation. Projeto usado como base para criação das APIs REST em Python.

ECLIPSE MOSQUITTO. MQTT Broker. Utilizado como broker local para comunicação IoT.

MYSQL. MySQL 8.0 Reference Manual. Base para modelagem relacional e comandos SQL.

JWT.IO. JSON Web Token Introduction. Base conceitual para autenticação baseada em token.

PYTHON. Python 3 Documentation. Linguagem utilizada para implementação dos microsserviços.